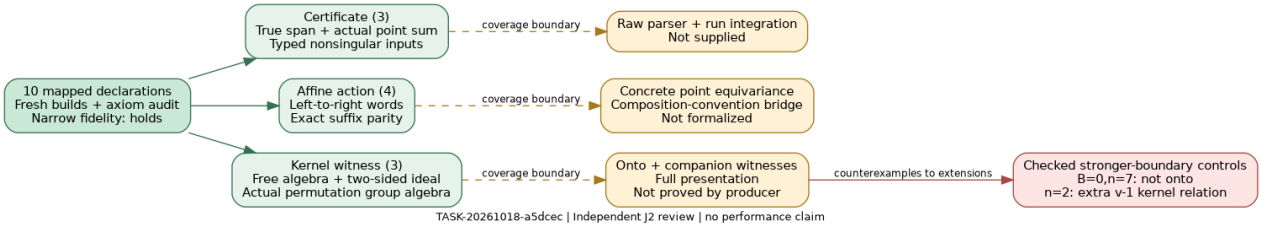
Independent review: certificate, affine action and kernel fidelity



Task **TASK-20261018-a5dcec**; owned joint **J2-certificate-affine-kernel-fidelity**; 2026-10-18.

Joint verdict: holds for the producer's ten explicitly scoped proof-map claims. All three frozen modules rebuilt, all ten named declarations were inspected and freshly axiom-audited, and the boundary controls passed. This is not approval of the whole retrospective or of the historical research conclusions. Literal coverage of all clauses in the three target specifications is **partial**: the scalar-to-point action bridge, surjectivity wording, companion witnesses and operational certificate-input integration require separate treatment.

The review did not read sibling reports or messages, change scientific status, edit producer sources or receipts, or commit. Every cited source is internal, retrieved from the frozen snapshot or the pinned local dependency checkout and verified by this reviewer. The complete actual read inventory, including files read only for hashing or searching, is in review_attestation.yaml and artifacts/sources-read.json.

Binding and machine evidence

The producer snapshot is crypto-autoresearcher@5e2bd6532fcf00730d8b231b55fe9427b7ae471d; the historical source revision named in the proof map is 6edd9e1ea14450fcb5a168137407ecf303e48773. Task authority is the precommitted handoff and review plan in /workspace/review-lean-evidence. The machine-readable assessment is claims.json.

All 198 frozen autoresearcher input files matched inputs.json SHA256 values. Before using the existing build cache, I checked that the three reviewed sources, lean-toolchain, lakefile.toml and lake-manifest.json matched the frozen files. The dependency checkout reports Mathlib 7fa22735254ddf7e11f119250a9f2d0352bc7c3a; Lean reports 4.35.0-rc1, commit 86c6347c75e39ec18c40e25ed2143b71a6e04a0a. Frozen hashes, cache binding, and fresh compilation results retain the checks.

The three exact frozen .lean modules were compiled directly into this task's output directory, using the pinned dependency cache read-only. run_lean.py records the invocation and environment; TypeAudit shows elaborated types; fresh axiom audit covers all ten names. Only propext, Classical.choice and Quot.sound occur; the sign-count lemma uses only propext. No sorry, admit, custom axiom, unsafe, or native_decide occurs in the reviewed proof sources or final control sources.

This is a fresh source compilation and Lean axiom audit, using cached imported dependencies. It is not an external lean4checker --fresh recheck or a rerun of the whole repository target runner; workflow review belongs to J3.

The ten mapped declarations

All names below have prefix CryptoResearch.. Every row has machine status passed, narrow fidelity faithful_scoped, and review outcome validated_scoped for its stated proof-map scope.

Declaration	What was actually checked	Boundary
Semaev.inDeclaredSpan_iff	Finite coefficient combinations equal Mathlib Submodule.span membership.	No independence or global spanning hypothesis is needed.
Semaev.checkDecomposition_iff	The Boolean checker is equivalent to equation, actual span, and actual point-group sum conditions.	Inputs are already certified nonsingular affine summands.

Declaration	What was actually checked	Boundary
Semaev.decidable_validDecomposition	A Boolean with the exact per-certificate correctness contract exists for every typed input.	Executable data comes from the separately supplied instance and checker, not the existential theorem alone.
NCP.wordSign_eq_negOne_pow	The recursive sign is minus one to the negation count.	Apply it to a suffix for the sign assigned to an occurrence.
NCP.word_action_affine_normal_form	Left-to-right scalar action equals signed affine form over any commutative ring.	The concrete curve action is not instantiated.
NCP.same_shadow_same_action	Equal integer sign and signed-count functions imply equal action functions.	It is a sufficient condition, not an injectivity or full-kernel statement.
NCP.orbit_value	The affine formula at scalar 1, in ZMod n.	No curve generator or experiment implementation is connected formally.
NCP.neg_square_not_in_commutators	The element v^2-1 is outside the genuine two-sided ideal of all commutators in the free integer algebra.	Evaluation at integer 2 is a valid separating homomorphism.
NCP.neg_square_in_action_kernel	The same element lies in the kernel of evaluation into the group algebra of actual affine permutations.	The map is into that algebra; surjectivity is not proved or needed.
NCP.kernel_not_generated_by_commutators	Those two genuine ideals are unequal, by the displayed witness.	No full presentation or companion (v*t_i)^2-1 theorem is supplied.

Certificate: mathematical validity and input boundary

Certificate.lean:13 defines membership by enumerating coefficient tuples from finite k. The equivalence at line 23 connects it to Mathlib's actual linear span; it is not a renamed user-supplied predicate. This works for an empty family and a dependent family. Finiteness of K is unnecessary once equality in K is decidable and the coefficient field is finite, so the generalized field assumptions do not weaken the finite-field target.

Certificate.lean:29 requires a proof of E.Nonsingular x y. Mathlib's definition includes the curve equation, and .point constructs the actual .some point. The equation conjunct is consequently redundant on this domain, but it is not circular with respect to span membership or the target sum. The sum in ValidDecomposition really uses E.Point addition; inspection of Mathlib's point addition confirms that this is its Weierstrass group law, not a declared addition table. The hypotheses are satisfiable, including in characteristic two.

The actual computational instance is at line 52, and the checker at line 59 uses it. The target's named theorem, at line 76, is an existential proposition. In isolation that proposition could be obtained classically for any proposition; it would not establish an executable verifier. Here the producer also supplies the genuine computational instance and the checker-to-span/group-law equivalence, and both were inspected. I found no instance/fixture substitution.

Kernel-checked controls in BoundaryControls.lean establish:

- Empty span membership is exactly x=0; an empty tuple accepts exactly the identity target.

- On y^2=x^3+1 over ZMod 7, (0,0) fails the equation and cannot inhabit AffineSummand; it is not silently accepted.

- (0,1) with empty declared span is accepted for its own target and rejected for target zero; (1,3) is rejected by the empty span despite a correct sum target.

- Actual doubling (0,1)+(0,1)=(0,6) is accepted, and the wrong target (0,1) is rejected.

- A dependent family [1,1] accepts 3 over ZMod 7.

- A nonsingular characteristic-two point on y^2+y=x^3 gives an accepted certificate.

The raw-coordinate control proves a typing boundary, not a Boolean parser rejection. No serialized-input parser or run-wrapper integration is provided. The frozen certificate target asks for a tuple of affine points of E(K), so the typed-domain decidability requirement is met. Any operational claim that arbitrary untrusted

coordinates are independently parsed and rejected, or that historical run certificates have been reverified by this checker, remains unproved. The producer explicitly disclaims the parser and runtime bounds. The much stronger Semaev prose about degree falls, attribution, novelty or growth is outside these three declarations.

Affine action: suffix parity and multiplication order

AffineNormalForm.lean:16 executes the head letter and then the tail. For a head translation, its suffix is precisely the tail, so the increment to its signed count is wordSign tail. A head negation contributes no translation and changes no right-hand suffix of the remaining occurrences. Together with wordSign_eq_negOne_pow, this derives the source's positional suffix-parity convention exactly.

With translation scalar 2, starting value 3, and modulus 7, the checked result is T;N -> 2, while N;T -> 6. The signed translation counts are respectively -1 and +1; two suffix negations give +1. Empty alphabets, empty words and characteristic two are harmless. In characteristic two signs can induce the same action, but no converse to equal-shadow-implies-equal-action is claimed.

There is an intentional convention boundary between this module and the group-algebra module. Standard permutation multiplication applies the right factor first. The checked multiplication_order_differs shows that the permutation product T*N sends 3 to 6, whereas left-to-right word [T,N] sends it to 2. The historical source explicitly discusses the anti-homomorphism convention; the proof map does not assert that these two raw evaluations coincide. The kernel witness v^2-1 is insensitive to reversing the word order. A general bridge must specify reversal/opposite multiplication or inversion/conjugation; it is not present in these files.

The scalar theorem gives the full affine identity mathematically in the canonical cyclic model. Transport to an arbitrary stated cyclic subgroup follows by a -> aP, but the formal equivariance and concrete implementation bridge remain absent. Thus the frozen affine target's scalar identity, parity and equal-shadow consequence are covered; its literal arbitrary-point action is not itself instantiated by the delivered declarations. The original note's shadow reachability iff, whole kernel presentations, value-set law, measured tables and direction closure are not certified by this batch.

Kernel witness: genuine algebra, incomplete broader coverage

CommutatorIdeal.lean:11 uses Mathlib FreeAlgebra over integers, with one generator for negation and B translation generators. Its ideal is TwoSidedIdeal.span of all algebra commutators. The integer evaluation kills every generator of this ideal and sends v^2-1 to 3; the containment in an actual kernel is proved using the span universal property. This is a genuine separation argument with no assumed abelianization result.

The target group is the closure inside permutations of ZMod n of negation and every translation. Evaluation uses the group-algebra basis embedding of these actual permutations. Negation squared is identity by function extensionality, so the witness is in the actual evaluation kernel, including degenerate n and B=0. These hypotheses are neither fake algebra axioms nor a preassumed kernel inequality.

The frozen kernel target says evaluation is **onto** the full affine group algebra and also includes companion witnesses. The producer explicitly drops surjectivity and the companions from its claim. My checked empty_evaluation_not_surjective uses B=0,n=7: every image centralizes negation, whereas the basis translation by 1 does not. Therefore the map cannot be onto. The same issue arises with a positive number of zero translations. Under the historical note's odd prime order, a nonzero translation and B>=1, surjectivity is mathematically recoverable, but it is not a universal fact about the delivered parameters. This does not invalidate the stronger-in-generality kernel inequality, whose proof needs no surjectivity.

The one witness proves the principal refutation, not the full target prose. The (v*t_i)^2-1 companion statements, full source kernel presentation and translations-only lattice presentation have no delivered formal proofs in these modules. They must not inherit the ten declarations' machine status.

An additional historical scope boundary is retained in EvenModulusBoundary.lean. For n=2, actual negation is identity, so v-1 is an additional kernel relation. It is outside the finite ideal generated by v^2-1, (v*t)^2-1 and t^2-1, as integer evaluation t->1,v->-1 sends it to -2 and kills all three generators. The same separating map kills every translation commutator and every translation-lattice binomial, so the written extension to the full historical generator list follows. This contradicts extending the full kernel presentation to literally “any n”; it does not contradict the source's local odd-prime setup or the producer's mapped inequality. The reduction to that full binomial list is a written derivation; the finite-ideal separation and actual extra kernel relation are Lean checked.

Controls, limitations and next obligations

BoundaryControls-attempt-07.log checks 24 control theorems and their axioms; EvenModulusBoundary-attempt-02.log checks two more. The isolated #eval outputs true for the doubling certificate; it is retained only as a compiled-execution smoke check. The mathematical verdict relies on the kernel-checked theorems, not native evaluation. Earlier invocation, elaboration and reduction failures are retained and are not mathematical counterexamples. Plain decide initially got stuck reducing a doubling calculation; unfolding the genuine addition and simplifying zero arithmetic supplied the final kernel proof. An initial wrong working-directory invocation and missing default toolchain were corrected with the specified root and explicit pinned toolchain. No producer source changed.

The remaining obligations are concrete:

1. Supply a raw-coordinate decision/parser contract and execution integration before claiming independent checking of serialized run certificates.

2. Formalize scalar-to-point equivariance and fix the composition convention in a bridge theorem before treating the scalar and group-algebra actions as one concrete implementation.

3. Prove surjectivity under suitable generating assumptions or remove “onto” from any universal target claim; add companion witnesses if every clause of that target is required.

4. Independently prove/review the full historical kernel, reachability and cost/closure claims; preserve odd-modulus restrictions where required.

No canonical scientific graph is changed by this review. The producer's proof map and README already mark the broader coverage open. Boundary diagram and PDF are derived views of this scoped review. Archival and any later official decision belong to the designated Coordinator task.

Review attestation

task_id: TASK-20261018-a5dcec; joints_owned: [J2-certificate-affine-kernel-fidelity]; read_sibling_reports: false; verdict: holds on the ten narrow mapped claims. Mathematical validity and narrow fidelity hold; full target and historical-source coverage are partial. This was an ordinary independent semantic review, not the round's blind rederivation. The full path inventory and precise read modes are retained in the accompanying YAML/JSON attestation. No sibling verdict was received or used.